TinkerWithMe

tinkerwithanne@gmail.com
Nairobi, Kenya

IoT Garden Monitor · Full Curriculum

2-session series (90 min each) · 10–14 yrs · Intermediate

Full facilitator curriculum **PREMIUM**

Overview

Build a sensor kit that keeps a real seedling alive, then automate its care.

DETAIL	
Track	Arduino Robotics & Electronics
Format	2-session series (90 min each)
Ages	10–14 yrs
Level	Intermediate
Price	from KES 6,000
Standalone	No
Prerequisites	Temperature logger (free, recommended)

Course Overview

IoT Garden Monitor is a two-session build in which each student assembles a real environmental sensing station, plants a living seedling in it, and then teaches the system to look after that seedling on its own. Session 1 is about "sensing the world" — wiring a soil moisture sensor, a DHT11 temperature/humidity sensor and an LDR light sensor, reading them correctly, and displaying live data on an LCD screen, before the seedling goes into its monitored pot. Session 2 is about "closing the loop" — turning raw sensor numbers into decisions, so a relay switches a small water pump when the soil gets dry, and a buzzer sounds when conditions drift outside a safe range.

The arc of the course mirrors how real IoT products are built: first you instrument something and prove your readings are trustworthy, then you automate a response to those readings. Students leave the first session with a working data logger they

can already narrate ("today my soil is at 40%, my light is low"), and leave the second session with an autonomous mini-greenhouse that keeps a living plant alive with no daily input from them.

The take-home is the entire physical kit — Arduino, breadboard, all sensors, the relay/pump assembly, and the actual seedling growing in its sensor-fitted pot — plus the two complete sketches so families can keep tinkering (and keep the plant alive) at home.

Learning Outcomes

- Wire and read three different sensor types (resistive/capacitive analog, digital protocol, and analog light) on a shared breadboard without pin conflicts.
- Use an I2C LCD to display multiple rotating readings, understanding why I2C saves pins compared to a parallel LCD.
- Explain the difference between raw analog values (0–1023) and calibrated, human-meaningful percentages, and calibrate a sensor against two known reference points.
- Write and modify conditional logic (thresholds, `if`/`else`) that turns sensor input into an actuator output.
- Safely wire and drive a relay module to switch a higher-current, separately-powered load (a water pump) from a low-current microcontroller pin.
- Debug a real circuit systematically: isolate whether a fault is in wiring, code, power, or sensor calibration.
- Care for a living plant using a self-built automated system, and articulate how the code's timing (cooldowns, run times) prevents overwatering.

Materials Master List

ITEM	SPEC	QTY PER STUDENT	NOTES
Microcontroller board	Arduino Uno R3 (or fully compatible clone)	1	Nerokas/Pixel Electronics stock standard Uno boards
USB cable	USB-A to USB-B	1	For programming and power
Breadboard	830 tie-point, full size	1	Needed for both sessions' extra components

Jumper wires	Male-to-male, assorted lengths, pack of 40	1 pack	
Jumper wires	Male-to-female, pack of 20	1 pack	For sensor modules with pin headers
Soil moisture sensor	Capacitive soil moisture sensor v1.2 (corrosion-resistant)	1	Capacitive preferred over resistive — doesn't corrode in soil over weeks
Humidity/ temperature sensor	DHT11 module (3-pin, breakout board with onboard pull-up)	1	Module version avoids a separate 10kΩ pull-up resistor
Light sensor	5mm LDR (photoresistor)	1	
Resistor	10kΩ, 1/4W	1	For LDR voltage divider
Display	16x2 LCD with I2C backpack (address 0x27 or 0x3F)	1	Check address with an I2C scanner sketch if blank
Relay module	5V single-channel relay board (opto-isolated, active-LOW)	1	Used in Session 2 only
Water pump	Mini submersible DC pump, 3–6V	1	Used in Session 2 only
Tubing	Silicone tubing, ~30cm, 4–5mm internal diameter	1 length	Connects pump outlet to pot
Buzzer	Active buzzer module (3-pin, with onboard driver)	1	Direct digital-pin drive, no transistor needed
Pump power supply	4xAA battery holder + 4 AA batteries (6V) OR a spare 5V USB supply	1	Powers pump/relay load side — keep separate from Arduino supply
Pot	Small plastic plant pot, 10–12cm, with drainage holes	1	
Seedling	Fast-growing seedling (bean, spinach or marigold), pre-germinated	1	Have 10% spares in case of damage
Potting soil	Small bag of potting mix	~500g	
Water reservoir	Small plastic cup or container (150–250ml)	1	Sits beside the pot; pump draws from here
Waterproof tray	Plastic tray or plate, larger than the pot	1	Keeps spills off the breadboard/Arduino
		a few	

Mounting tape / zip ties	Double-sided foam tape or small zip ties		To mount LCD and secure sensor cables
Marker + labels	Fine-tip marker, small sticky labels	1 set shared	For a watering log and cable labelling
Small screwdriver	Flat-head, for relay screw terminals (if present)	1 shared per table	
Laptop	Arduino IDE 1.8+ or 2.x installed, DHT and LiquidCrystal_I2C libraries pre-installed	1 per student or pair	Install libraries before Session 1 begins

Session Plans

Session 1: Sense — Soil, Air & Light, Displayed Live

Objectives

- Wire and read a capacitive soil moisture sensor, a DHT11, and an LDR on one shared Arduino, without pin or power conflicts.
- Display live, rotating sensor readings on an I2C LCD.
- Calibrate the soil sensor and pot a real seedling with the probe correctly placed.

Timing (90 minutes)

- **0–10 min:** Hook — ask "how does a plant tell you it's thirsty?" Show a wilted vs healthy plant photo. Introduce the idea: today we build the plant's "senses."
- **10–20 min:** Materials distribution and safety briefing — handling the breadboard, never forcing pins, keeping the DHT11 dry, general electrostatic care.
- **20–35 min:** Guided wiring — facilitator wires one channel at a time on a demo board (LCD first, then soil sensor, then DHT11, then LDR), students copy each step before moving to the next.
- **35–55 min:** Open Arduino IDE together, paste and explain the sketch section by section (analogRead, map, DHT library calls, LCD calls), then upload.
- **55–70 min:** Test and troubleshoot — every student must see live numbers on their LCD before moving on; facilitator circulates with a checklist.
- **70–85 min:** Calibrate the soil sensor (dry air vs a cup of water), note the two raw values, then plant the seedling and insert the probe to the correct depth (fully into soil, never touching the pot wall).

- **85–90 min:** Clean-up, label kits with names, preview Session 2 ("next time your plant waters itself").

Step-by-step

1. Open by asking students to name what a plant needs (water, light, warmth) and map each need to a sensor (soil moisture, LDR, DHT11).
2. Demonstrate the breadboard power rails: run a red jumper from Arduino 5V to the breadboard's red rail, and a black jumper from Arduino GND to the black rail. Explain every sensor's VCC/GND will tap these rails.
3. Wire the LCD first (so students always have visible feedback): VCC/GND to rails, SDA to A4, SCL to A5.
4. Wire the soil sensor: VCC/GND to rails, AO (analog out) to A0.
5. Wire the LDR voltage divider: one leg of the LDR to the 5V rail, the other leg to A1 and to one leg of the 10kΩ resistor; the resistor's other leg to GND. Explain that as light increases, the LDR's resistance drops, so voltage at A1 changes — this is a classic voltage divider.
6. Wire the DHT11 module: VCC/GND to rails, DATA to D2.
7. Live-code together: type or paste the Session 1 sketch, walking through what `map()` does and why raw analog values need calibrating into percentages.
8. Upload, open Serial Monitor and the LCD together — confirm both show sensible values (soil % should move when you touch the probe with a wet finger; light % should change when you cover the LDR).
9. Calibration exercise: hold the soil probe in dry air, note the raw value (should be near 1023); dip in a cup of water, note the raw value (should drop toward 300–400). Have each student write both numbers on a sticky label on their board.
10. Update the sketch's `map()` arguments to each student's own two calibration numbers if they differ noticeably from the defaults.
11. Plant the seedling: fill the pot with potting soil, transplant the seedling, then push the soil sensor probe fully into the soil near the roots (not touching the pot's plastic wall, not sitting on the surface).
12. Mount the LCD to the outside of the pot or tray with tape so it's visible without disturbing wiring.

Build detail — Materials

- Arduino Uno, USB cable, breadboard, jumper wires (M-M and M-F)

- 16x2 I2C LCD, capacitive soil moisture sensor, DHT11 module, LDR, 10kΩ resistor
- Pot with drainage holes, potting soil, seedling, waterproof tray

Build detail — Wiring (component → pin)

- LCD I2C module VCC → 5V rail; GND → GND rail; SDA → Arduino A4; SCL → Arduino A5
- Soil moisture sensor VCC → 5V rail; GND → GND rail; AO → Arduino A0
- LDR leg 1 → 5V rail; LDR leg 2 → Arduino A1 and → 10kΩ resistor leg 1; resistor leg 2 → GND rail
- DHT11 module VCC → 5V rail; GND → GND rail; DATA (OUT/SIG) → Arduino D2

Build detail — Complete Arduino sketch

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

// ---- Pin setup ----
#define DHTPIN 2       // DHT11 data pin
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

LiquidCrystal_I2C lcd(0x27, 16, 2); // change to 0x3F if your LCD doesn't respond

const int soilPin = A0;   // soil moisture sensor analog pin
const int ldrPin  = A1;   // LDR voltage divider analog pin

// ---- Calibration values ----
// Update these after doing the dry-air / wet-cup test with YOUR sensor
const int soilDryValue = 1023; // raw reading in dry air (0% moisture)
const int soilWetValue = 300;  // raw reading dipped in water (100% moisture)

unsigned long lastRead = 0;
const unsigned long interval = 2000; // read every 2 seconds
int displayMode = 0; // toggles which two readings show on the LCD

void setup() {
  Serial.begin(9600);
  lcd.init();
  lcd.backlight();
  dht.begin();

  lcd.setCursor(0, 0);
  lcd.print("Garden Monitor");
  lcd.setCursor(0, 1);
  lcd.print("Booting up...");
  delay(2000);
  lcd.clear();
}
```

```
void loop() {
  if (millis() - lastRead >= interval) {
    lastRead = millis();

    // --- Read soil moisture and convert to a 0-100% scale ---
    int soilRaw = analogRead(soilPin);
    int soilPercent = map(soilRaw, soilDryValue, soilWetValue, 0, 100);
    soilPercent = constrain(soilPercent, 0, 100);

    // --- Read light level and convert to a 0-100% scale ---
    int lightRaw = analogRead(ldrPin);
    int lightPercent = map(lightRaw, 0, 1023, 0, 100);

    // --- Read temperature and humidity from DHT11 ---
    float temp = dht.readTemperature();
    float hum  = dht.readHumidity();

    // --- Print everything to Serial Monitor for debugging ---
    Serial.print("Soil: ");
    Serial.print(soilPercent);
    Serial.print("%  Light: ");
    Serial.print(lightPercent);
    Serial.print("%  Temp: ");
    Serial.print(temp);
    Serial.print("C  Humidity: ");
    Serial.print(hum);
    Serial.println("%");

    // --- Show readings on the LCD, alternating between two screens ---
    lcd.clear();
    if (displayMode == 0) {
      lcd.setCursor(0, 0);
      lcd.print("Soil: ");
      lcd.print(soilPercent);
      lcd.print("%");

      lcd.setCursor(0, 1);
      lcd.print("Humidity: ");
      lcd.print((int)hum);
      lcd.print("%");
    } else {
      lcd.setCursor(0, 0);
      lcd.print("Temp: ");
      lcd.print(temp);
      lcd.print("C");

      lcd.setCursor(0, 1);
      lcd.print("Light: ");
      lcd.print(lightPercent);
      lcd.print("%");
    }
    displayMode = 1 - displayMode; // flip between 0 and 1
  }
}
```

Checks for understanding

- "Why do we need to convert the soil sensor's raw number (0–1023) into a percentage? What are we actually calibrating against?"
- "What would happen to the LDR reading if you cupped your hand over it? Why?"
- "Why does the LCD use only two wires (SDA/SCL) instead of the many pins a basic LCD needs?"

Common mistakes & fixes

MISTAKE	SYMPTOM	FIX
LCD address wrong	LCD backlight on but blank, or garbled text	Run an I2C scanner sketch to find the true address (commonly 0x27 or 0x3F) and update the code
DHT11 wired to wrong pin or loose jumper	Serial shows "nan" for temp/humidity	Reseat DATA wire on D2, confirm VCC/GND are firm, wait a few seconds — DHT11 reads slowly
LDR wired without the resistor divider	Reading is always 0 or always 1023, never changes	Confirm the 10kΩ resistor is between A1's junction and GND, not missing
Soil sensor left uncalibrated	Percentage always shows 0% or negative/over 100%	Redo the dry-air/wet-cup test and update soilDryValue/soilWetValue in code
Soil probe touching pot's plastic wall	Reading stays high (dry) even after watering	Reposition probe fully into the soil mass near the roots

Session 2: Automate — Thresholds, Pump & Buzzer

Objectives

- Add a relay-driven water pump that responds automatically to low soil moisture.
- Add a buzzer that alerts on out-of-range temperature or flooded soil.
- Understand and tune threshold values, run-times and cooldowns to avoid overwatering.

Timing (90 minutes)

- **0–10 min:** Recap Session 1 readings; ask "if you weren't here for a week, how would this plant survive?" Introduce automation as the answer.
- **10–20 min:** Safety briefing on mixing water and electronics — separate battery pack for the pump, relay as the safe boundary, waterproof tray, never touching wiring with wet hands.

- **20–40 min:** Guided wiring of relay, pump (with separate battery pack) and buzzer.
- **40–60 min:** Walk through the Session 2 sketch — thresholds, relay logic (active-LOW), pump run-time, cooldown timer, buzzer conditions. Upload.
- **60–75 min:** Live test: manually dry the soil test area or lift the probe to trigger the pump; cup a hand over the DHT11 to raise apparent temperature and confirm the buzzer fires; confirm pump auto-stops after its run time.
- **75–85 min:** Tune thresholds together for their specific plant/season; discuss why cooldown times would be much longer in real deployment (hours, not seconds).
- **85–90 min:** Showcase circle and take-home briefing (watering log, care instructions).

Step-by-step

1. Recap the four readings from Session 1 and ask students to predict what "automatic" behaviour each one could trigger.
2. Explain the relay as an electrically-isolated switch: the Arduino's small signal (D7) controls a coil that closes a much bigger circuit (pump + separate battery pack) without the Arduino ever touching the pump's power directly.
3. Wire the relay module: VCC/GND to the 5V/GND rails, IN to D7.
4. Wire the pump circuit on the relay's output side: battery pack positive → relay COM; relay NO (normally open) → pump positive wire; pump negative wire → battery pack negative. This circuit is entirely separate from the Arduino's own power.
5. Place the pump in the water reservoir cup, and run the silicone tubing from the pump outlet to just above the soil surface in the pot.
6. Wire the buzzer: positive lead to D8, negative lead to GND rail.
7. Live-code together: paste the Session 2 sketch, explain each threshold constant, why the relay code uses LOW to mean "on" (most low-cost relay boards are active-LOW), and how `pumpRunTime` and `pumpCooldown` prevent flooding.
8. Upload and test: dry the soil sensor area with a tissue or briefly lift it out of soil to simulate dryness, confirm the pump activates and the LCD shows "PUMP", then confirm it switches off on its own after 3 seconds.

9. Test the buzzer: warm the DHT11 with cupped hands or breath to push temperature above the high-alert threshold, confirm the buzzer sounds and the LCD shows an alert message.
10. As a group, discuss and adjust `soilThresholdPercent`, `tempHighAlert` and `tempLowAlert` to sensible values for their actual seedling and classroom conditions.
11. Reassemble the full kit neatly on the waterproof tray, secure all cables, and do a final "walk test" leaving it running for a few minutes unattended.

Build detail — Materials

- Everything from Session 1 (already wired), plus: relay module, mini submersible pump, silicone tubing, active buzzer, 4xAA battery pack (or spare 5V supply) for the pump circuit

Build detail — Wiring (component → pin)

- Relay module VCC → 5V rail; GND → GND rail; IN → Arduino D7
- Battery pack (+) → Relay COM terminal
- Relay NO terminal → Pump positive wire
- Pump negative wire → Battery pack (–)
- Buzzer module (+) → Arduino D8; Buzzer (–) → GND rail
- *(Soil sensor, LDR, DHT11 and LCD wiring remain exactly as in Session 1 — do not rewire these.)*

Build detail — Complete Arduino sketch

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

// ---- Pin setup ----
#define DHTPIN 2
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

LiquidCrystal_I2C lcd(0x27, 16, 2); // change to 0x3F if needed

const int soilPin   = A0;
const int ldrPin    = A1;
const int relayPin  = 7;  // controls the pump
const int buzzerPin = 8;  // sounds alerts

// ---- Calibration (carry over your Session 1 values) ----
const int soilDryValue = 1023;
const int soilWetValue = 300;
```

```
// ---- Automation thresholds (tune these with your class) ----
const int soilThresholdPercent = 35;   // pump turns on if soil moisture drops below this
const int soilOverwetPercent   = 85;   // buzzer warns if soil is flooded above this
const float tempHighAlert = 32.0;      // buzzer warns above this temperature (C)
const float tempLowAlert  = 15.0;      // buzzer warns below this temperature (C)

// ---- Timing controls ----
const unsigned long pumpRunTime  = 3000;   // pump runs for 3 seconds per watering
const unsigned long pumpCooldown = 60000;  // wait 60s between waterings
                                            // (in real deployment, set this to several HOURS,
                                            // e.g. 21600000UL for 6 hours, to avoid overwatering)
const unsigned long readInterval = 2000;   // how often we check sensors

unsigned long lastRead = 0;
unsigned long lastPumpTime = 0;
bool pumpRunning = false;
unsigned long pumpStartTime = 0;
int displayMode = 0;

void setup() {
  Serial.begin(9600);
  pinMode(relayPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);

  // Most low-cost relay boards are ACTIVE-LOW: LOW turns the relay ON.
  digitalWrite(relayPin, HIGH); // start with pump OFF
  digitalWrite(buzzerPin, LOW); // start with buzzer OFF

  lcd.init();
  lcd.backlight();
  dht.begin();

  lcd.setCursor(0, 0);
  lcd.print("Auto Garden");
  lcd.setCursor(0, 1);
  lcd.print("Starting...");
  delay(2000);
  lcd.clear();
}

void loop() {
  unsigned long now = millis();

  // --- Turn the pump off automatically once its run time is finished ---
  if (pumpRunning && (now - pumpStartTime >= pumpRunTime)) {
    digitalWrite(relayPin, HIGH); // relay OFF
    pumpRunning = false;
    lastPumpTime = now; // start the cooldown timer from here
  }

  if (now - lastRead >= readInterval) {
    lastRead = now;

    // --- Read all sensors ---
    int soilRaw = analogRead(soilPin);
    int soilPercent = map(soilRaw, soilDryValue, soilWetValue, 0, 100);
    soilPercent = constrain(soilPercent, 0, 100);

    int lightRaw = analogRead(ldrPin);
    int lightPercent = map(lightRaw, 0, 1023, 0, 100);
```

```
    float temp = dht.readTemperature();
    float hum  = dht.readHumidity();

    Serial.print("Soil:"); Serial.print(soilPercent);
    Serial.print("% Temp:"); Serial.print(temp);
    Serial.print("C Hum:"); Serial.print(hum);
    Serial.print("% Light:"); Serial.println(lightPercent);

    // --- Decide whether the plant needs watering ---
    bool needsWater = (soilPercent < soilThresholdPercent);
    bool canWater = (now - lastPumpTime >= pumpCooldown) && !pumpRunning;

    if (needsWater && canWater) {
      digitalWrite(relayPin, LOW); // relay ON (active-LOW)
      pumpRunning = true;
      pumpStartTime = now;
      Serial.println(">> Pump ON: watering seedling");
    }

    // --- Decide whether to sound the buzzer ---
    bool tempAlert  = (temp > tempHighAlert) || (temp < tempLowAlert);
    bool floodAlert = (soilPercent > soilOverwetPercent);

    if (tempAlert || floodAlert) {
      digitalWrite(buzzerPin, HIGH);
    } else {
      digitalWrite(buzzerPin, LOW);
    }

    // --- Update the LCD, alternating between two screens ---
    lcd.clear();
    if (displayMode == 0) {
      lcd.setCursor(0, 0);
      lcd.print("Soil:");
      lcd.print(soilPercent);
      lcd.print("% ");
      lcd.print(pumpRunning ? "PUMP" : "");

      lcd.setCursor(0, 1);
      lcd.print("Hum:");
      lcd.print((int)hum);
      lcd.print("% T:");
      lcd.print((int)temp);
      lcd.print("C");
    } else {
      lcd.setCursor(0, 0);
      lcd.print("Light:");
      lcd.print(lightPercent);
      lcd.print("%");

      lcd.setCursor(0, 1);
      if (tempAlert)       lcd.print("ALERT: Temp!");
      else if (floodAlert) lcd.print("ALERT: Flood!");
      else                 lcd.print("Status: OK");
    }
    displayMode = 1 - displayMode;
  }
}
```

Checks for understanding

- "Why does the relay code write LOW to turn the pump ON instead of HIGH? What would happen if we wired it assuming the opposite?"
- "What two jobs does `pumpCooldown` do for the plant's health, and why would we make it much longer than 60 seconds in a real deployment?"
- "If the buzzer keeps sounding, what are the two different sensor conditions that could be causing it, and how would you check which one it is?"

Common mistakes & fixes

MISTAKE	SYMPTOM	FIX
Relay wired backwards (HIGH/LOW logic assumed wrong)	Pump runs constantly except when it should be on	Confirm your relay board's active-LOW/active-HIGH type; swap HIGH/LOW in code to match
Pump and Arduino sharing the same battery incorrectly	Arduino resets or LCD flickers when pump starts	Keep the pump's power supply fully separate; only the relay's coil side connects to the Arduino
pumpCooldown set too short for the demo	Pump seems to "chatter" on and off repeatedly	This is expected in the demo (60s); explain that real deployment needs hours-long cooldowns
Buzzer wired to wrong pin or is a passive buzzer	No sound, or only a faint click	Confirm it's an active buzzer module (has its own driver chip) and is on D8
Tubing not seated in the reservoir or pot	Pump runs but no water reaches the soil	Check tubing is submerged at the pump end and positioned over soil (not pot rim) at the other end

Assessment & Showcase

Assessment is formative and hands-on across both sessions rather than written. In Session 1, "done" means every student's LCD is showing live, sensible soil/humidity/temperature/light readings that visibly respond when a sensor is touched, covered, or breathed on, and their seedling is planted with the probe correctly placed. In Session 2, "done" means the pump reliably switches on when soil is artificially dried and switches off on its own, the buzzer correctly distinguishes at least one alert condition, and the student can explain in their own words what each threshold in the code controls.

Close Session 2 with a five-minute showcase circle: each student places their finished kit on the table, briefly narrates what their plant's current readings mean, and deliberately triggers one automated behaviour for the group to see (e.g., lifting the soil probe to make the pump fire, or warming the DHT11 to sound the buzzer). Facilitators should note down, per student, which of the three sensors and which of the two automations were successfully demonstrated — this becomes the basis of a simple checklist sent home with families summarising what their child built and how to keep it running.

Extension & Home Challenges

- **Easy:** Add a third LCD screen (displayMode 2) that shows the raw soil sensor value alongside the calibrated percentage, so you can see exactly how the calibration math works in real time.
- **Medium:** Add a push button that lets you manually trigger a "water now" cycle at any time, overriding the cooldown once per press (hint: use a debounced button read and a boolean flag that bypasses the `canWater` check for one cycle).
- **Hard:** Log every watering event with a timestamp using `millis()` converted to minutes since power-on, store the last five watering times in an array, and display "last watered Xmin ago" on the LCD — then extend it further to only allow watering during a defined "daytime" window based on the LDR's light reading, so the pump never runs at night.